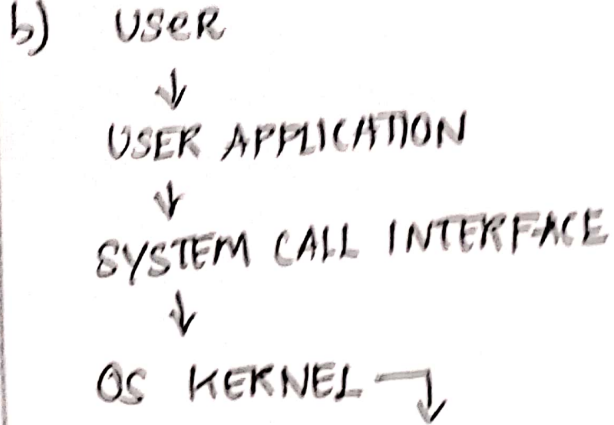


SHORT ANSWER ANALYSIS

1. Suppose a modern operating system supports multiple users and processes with shared resources like CPU, memory and I/O devices.
 - a) Classify different types of operating systems suitable for this scenario and justify your choice.
 - b) Draw the structure of an OS showing how resources are managed.
 - c) Write a sequence of system calls and execute a new process.
- 2) Consider an operating system managing multiple files for concurrent users
 - a) Categorize the system calls used for file and device
 - b) Illustrate how the OS handles file operations
 - c) Write a pseudo code to open, read, close a file.
- 3) Assume a real time operating system used in an embedded system controlling industrial machinery.
 - a) Identify type of OS and explain its suitability.
 - b) Draw a diagram representing the interaction between hardware, kernel
 - c) Tell what system calls are required for synchronization.

4. A distributed computing system consists of multiple interconnected computers sharing resources over a network.
- clarify the type of operating system used and compare it with centralized OS.
 - Architecture of distributed OS showing communication between nodes.
 - Write the sequence of system calls involved in IPC.
5. Consider an scenario where an OS must manage process scheduling, memory allocation and device communication effectively.
- explain the major functions of OS in handling these jobs.
 - represent a block diagram of OS and interactions.
 - write system call example for memory and device.
- Solutions:-
- 1) A) * Multitasking operating system can be suitable for this scenario as it can handle multiple jobs at a time without any lag or bug, multiuser OS can support multiple users using the single system at same time.
- * A Time sharing multiuser operating system is the most suitable because it effectively shares the CPU, memory, I/O devices among multiple users.



PROCESS MANAGEMENT

MEMORY

FILE

I/O

SECURITY AND PROTECTION

c) `fork()` - creates a child process

`exec()` - loads and executes a new program

`wait()` - parents wait for the child process to complete

`exit()` - child process terminates.

2. a) `open()` - opens a file.

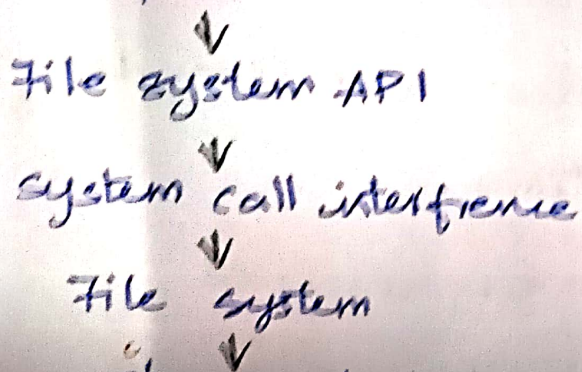
`read()` - reads data.

`write()` - writes data to a device

`ioctl()` - controls device specific operations.

`mount()` - mounts a storage device.

b) User Application

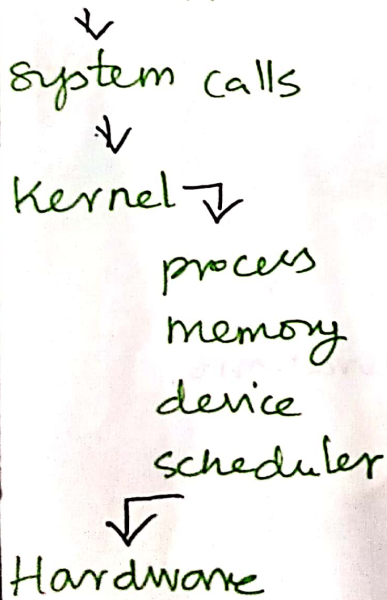


C START
Open file
read file data
Process data
close file
END

3) A) Real Time Operating System (RTOS)

- * provides deterministic response
- * ensures high reliability
- * handles time critical tasks
- * Supports low latency.
- * prevents system failure

B) User Application



C) * Synchronize multiple processes and threads.

- * prevents race conditions.
- * Control access to shared resources
- * ensure data consistency.

4) a) distributed operating System

- * multiple interconnected computers.
- * Resource sharing
- * load balancing
- * High Availability.

c) Sequence:

Application $\rightarrow$ socket() $\rightarrow$ connect() $\rightarrow$ send() $\rightarrow$
receive() $\rightarrow$ close().

5. a) process scheduling:

- * selects the next process to execute.
- * Improves CPU utilization.

b) memory Allocation:

- * Allocates and deallocates
- * supports virtual memory

c) device communication:

- * control I/O
- * use device drivers

System calls:

Memory allocation:

* `brk()`

* `sbrk()`

* `map()`

* `mmap()`

* Device management calls:

* `open()`

* `read()`

* `write()`

* `ioctl()`

* `close()`